Formação Completa

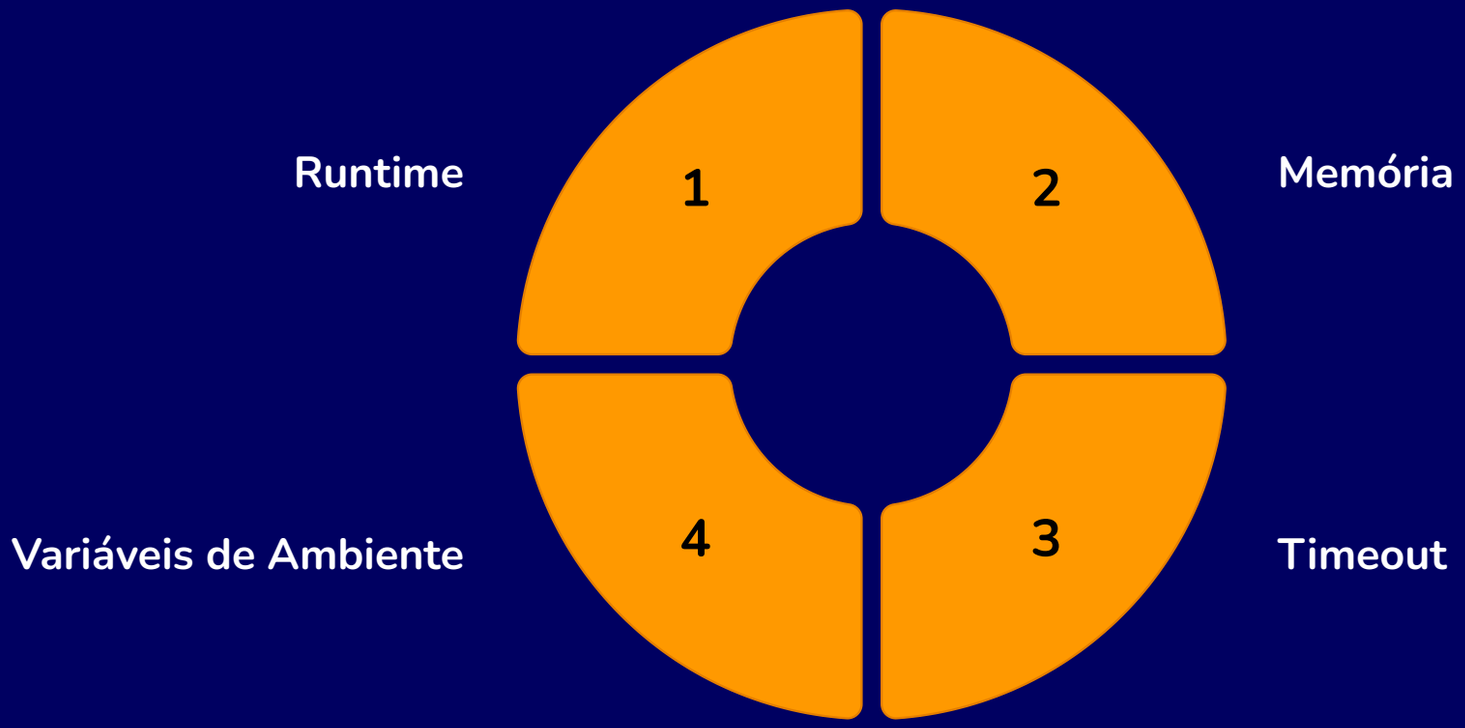
AWS Developer Associate

Desenvolvendo e Configurando AWS Lambda



Configurações Essenciais da Função Lambda

As configurações de uma função Lambda definem seu comportamento em tempo de execução. Escolher as configurações certas é crucial para garantir eficiência e otimização de custos.



Vamos detalhar cada uma dessas configurações a seguir.

Escolhendo o Runtime Adequado

Especifica a linguagem de programação e versão utilizada pela função.

Compatibilidade

Garanta que o runtime escolhido seja compatível com as bibliotecas e dependências do seu código.

Verifique o ciclo de vida do runtime para garantir que ele ainda seja suportado pela AWS.

Performance

Algumas linguagens têm desempenho melhor em cenários específicos.

Teste diferentes runtimes para encontrar o mais adequado para sua aplicação.

Linguagens Suportadas

- Python
- Node.js
- Java
- Go
- .NET
- Ruby

Configuração de Memória

128MB

Mínimo

Valor mínimo de memória que pode ser alocado para uma função Lambda.

10GB

Máximo

Valor máximo de memória exato 10.240 MB, que pode ser alocado para uma função Lambda, , em incrementos de 1 MB

ms

Faturamento

A duração é calculada do início à conclusão da execução, arredondada para 1 ms* mais próximos. O preço depende da memória alocada, que ajusta proporcionalmente a CPU e outros recursos.



A memória alocada afeta diretamente a CPU disponível para a função. Aumentar a memória pode acelerar a execução, mas também aumenta o custo.

Fonte: https://docs.aws.amazon.com/pt_br/lambda/latest/dg/configuration-memory.html

Configuração de Timeout

Definição

Período máximo que uma função Lambda pode executar antes de ser interrompida automaticamente.

Configuração

Valor padrão de 3 segundos, ajustável em incrementos de 1 segundo.

Limite Máximo

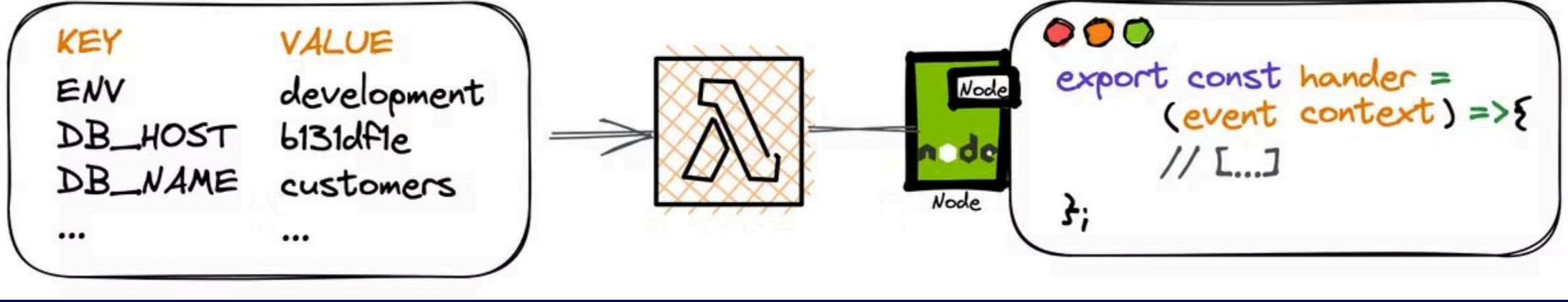
Até 900 segundos (15 minutos) por execução.

Considerações

Configure de acordo com a complexidade da tarefa para evitar cobranças desnecessárias ou interrupções prematuras.

O timeout adequado evita falhas em operações longas e otimiza custos em funções rápidas. Se a função exceder o timeout, ela será interrompida e um erro será gerado.

Monitore o tempo de execução real para ajustar este parâmetro com precisão e para identificar possíveis gargalos.



Variáveis de Ambiente

Variáveis de ambiente são pares chave-valor utilizados para armazenar informações separadas do código-fonte.

Flexibilidade

Altere configurações como endpoints de API, chaves de acesso ou modos de operação sem reimplantar código. Adapte o comportamento para diferentes ambientes.

Segurança

Armazene informações sensíveis separadas do código-fonte, evitando exposição em repositórios. Use AWS KMS para criptografar informações sensíveis.

Runtime

O Lambda disponibiliza as variáveis automaticamente para seu código durante a execução.

Contexto

Variáveis adicionais fornecem informações sobre a função e a solicitação de invocação.

Exemplo: Variáveis de Ambiente

1. Criar uma função Lambda no console da AWS
2. Definir a variável DATABASE_URL com um valor
3. Testar a função e verificar o retorno
4. Alterar o valor da variável e testar novamente

```python
import os

def lambda_handler(event, context):
    # Obtém o valor da variável de ambiente "DATABASE_URL"
    database_url = os.getenv("DATABASE_URL", "default_value")
    return {"statusCode": 200, "body": f"Database URL: {database_url}"}
```

Conclusão: Otimizando Configurações do AWS Lambda

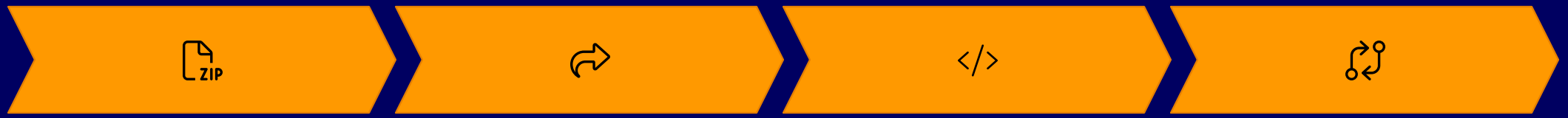
A combinação adequada de runtime, memória, timeout e variáveis de ambiente é essencial para maximizar a eficiência das suas funções Lambda.

Estas configurações, quando ajustadas corretamente, garantem performance otimizada, menor custo operacional e adaptabilidade às necessidades específicas do seu workload.

Mantenha o monitoramento constante destes parâmetros para identificar oportunidades de melhoria contínua na sua infraestrutura serverless.



Camadas (Layers) para Reutilização de Código



Pacote ZIP

Arquivo compactado que contém código adicional, bibliotecas ou dados para suas funções.

Compartilhamento

Reutilize dependências e código comum entre múltiplas funções Lambda.

Conteúdo

Bibliotecas, runtimes personalizados e arquivos de configuração ficam disponíveis para suas funções.

Versionamento

Gerencie e atualize dependências sem modificar o código da função principal.

As Layers simplificam a manutenção de código ao separar dependências da lógica de negócios. Isso resulta em pacotes de implantação menores e desenvolvimento mais ágil.

Camadas (Layers) para Reutilização de Código



Processo de Criação de Camadas

1 **Criar Diretório**

Organize as dependências da sua camada em um diretório estruturado.

2 **Instalar Dependências**

Use gerenciadores de pacotes como pip ou npm para instalar as dependências.

3 **Criar Arquivo ZIP**

Compacte o diretório seguindo a estrutura exigida pelo runtime.

4 **Publicar Camada**

Publique a camada no AWS Lambda usando console, CLI ou SDK.

Criando um Layer para incluir bibliotecas Pandas e NumPy em uma função Lambda

```bash
#!/bin/bash

# Nome da camada
LAYER_NAME="MyLayer"
ZIP_FILE="my-layer.zip"
PYTHON_DIR="python"

# 1. Crie o diretório para a camada (se não existir)
mkdir -p "$PYTHON_DIR"

# 2. Instale as dependências no diretório
# Verifique se o pip está instalado
if ! command -v pip &> /dev/null; then
    echo "pip não encontrado. Instale o pip antes de executar o script."
    exit 1
fi

pip install --target "$PYTHON_DIR" pandas numpy

# 3. Crie o arquivo ZIP da camada
zip -r "$ZIP_FILE" "$PYTHON_DIR"

# 4. Publique a camada no AWS Lambda
aws lambda publish-layer-version \
    --layer-name "$LAYER_NAME" \
    --zip-file "fileb://$ZIP_FILE" \
    --compatible-runtimes python3.9

echo "Camada $LAYER_NAME publicada com sucesso!"
```

Gerenciamento de Versões de Camadas

Versionamento

Camadas têm versões imutáveis, permitindo atualizações sem afetar funções existentes.

Cada publicação cria uma nova versão com ARN único.

Permissões

Camadas podem ser compartilhadas entre contas AWS.

Configure permissões para controlar quem pode usar suas camadas.

Boas Práticas

- Documente dependências incluídas
- Teste camadas antes de publicar
- Mantenha versões antigas para compatibilidade

Gerenciamento de Concorrência (Concurrency)

Definição

Número de instâncias da função Lambda executando simultaneamente. Cada solicitação cria uma nova instância até o limite de concorrência.

Concorrência Reservada (Reserved Concurrency)

Limita o número máximo de instâncias da função que podem executar simultaneamente. Protege outros sistemas e funções.

Concorrência Provisionada (Provisioned Concurrency)

Garante que um número específico de instâncias estejam sempre prontas. Reduz cold starts, mas aumenta custos.



Configurando Concorrência Reservada: Via Console AWS

1. Acesse a função Lambda
2. Clique em "Configurações"
3. No menu à esquerda, selecione Detecção de Recursão e Simultaneidade.
4. Na seção Simultaneidade, localize o botão Editar no canto direito e clique nele.
5. Marque a opção Reservar simultaneidade.
6. Digite a quantidade desejada no campo correspondente.
7. Clique em Salvar para aplicar as alterações.

A concorrência reservada garante que sua função tenha sempre capacidade disponível, mesmo durante picos de tráfego em outras funções da conta.

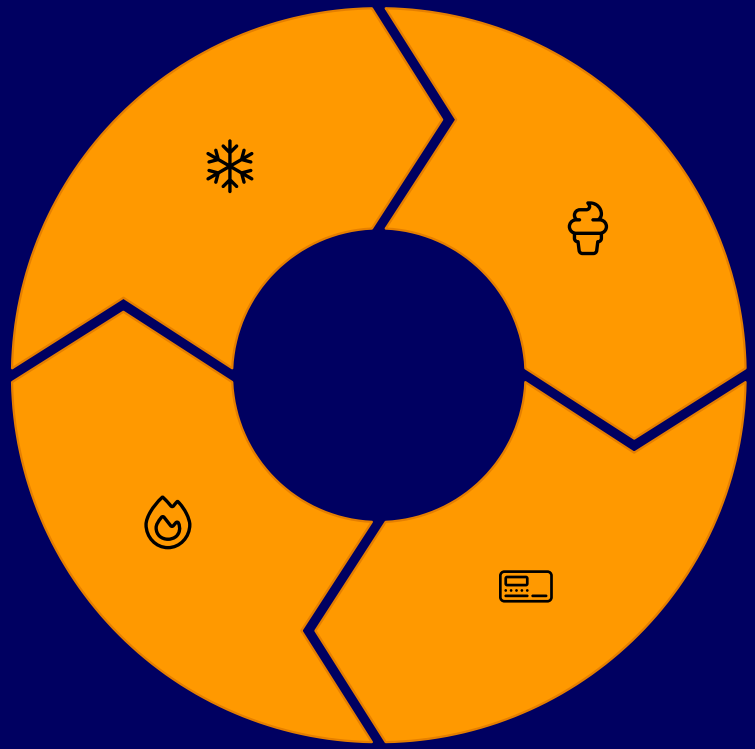
Configurando Concorrência Reservada: Via AWS CLI

```
aws lambda put-function-concurrency \
  --function-name MinhaFuncao \
  --reserved-concurrent-executions 10
```

Cold Starts

O que são

Atraso na primeira execução ou após inatividade.



Causas

Inicialização do ambiente e carregamento do código.

Mitigação

Técnicas para reduzir ou eliminar o atraso.

Impacto

Latência adicional na resposta da função.

Mitigando Cold Starts

Conexões Persistentes

Reutilize conexões com bancos de dados e outros serviços entre invocações.

Concorrência Provisionada

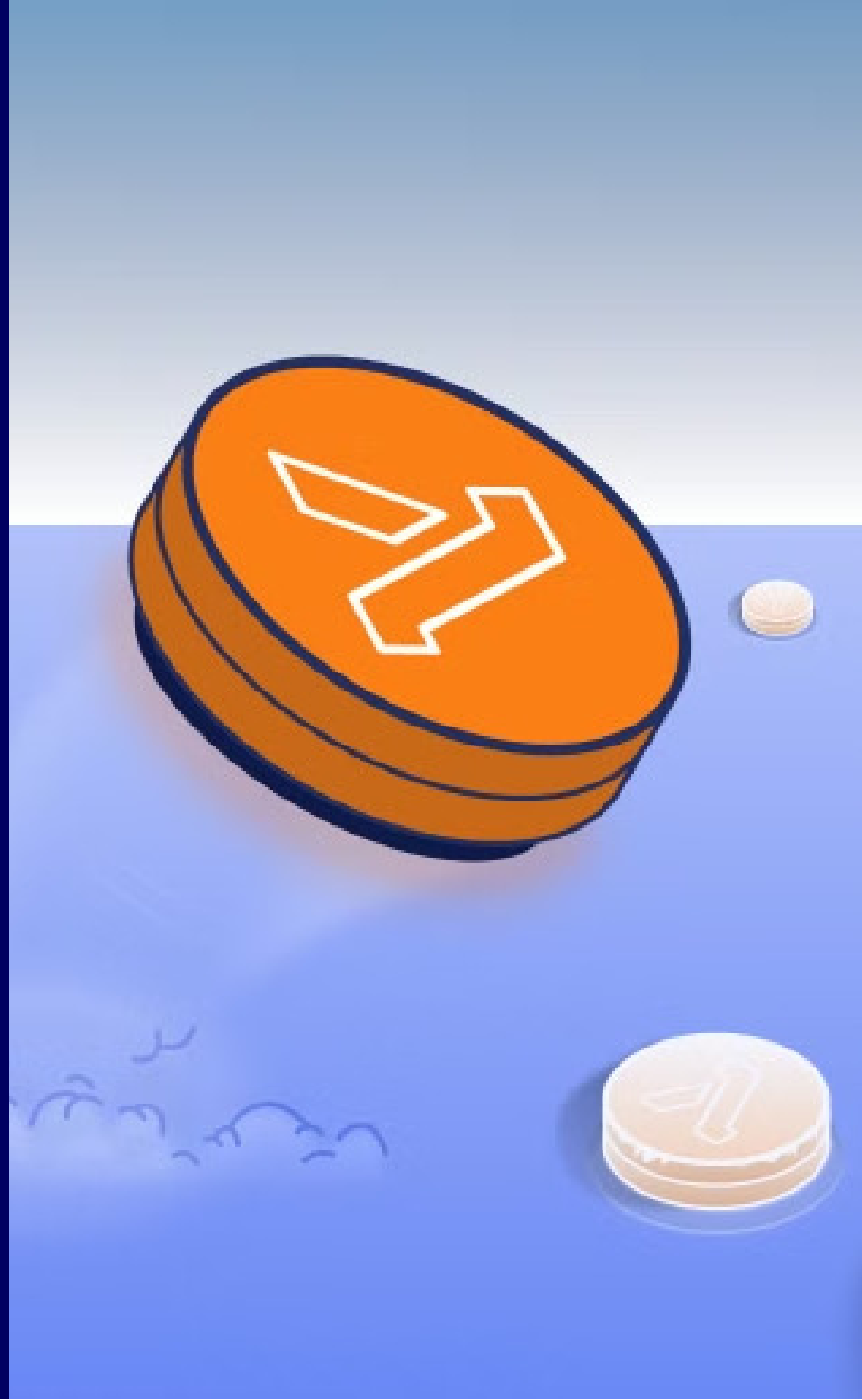
Mantenha instâncias pré-inicializadas prontas para responder.

Keep-alive

Envie invocações periódicas para manter a função "aquecida".

Aumentar Memória

Mais memória significa mais CPU, acelerando a inicialização.



Persistência de Conexões

Conceito Básico

Manter conexões abertas entre invocações da função Lambda.

Elimina o overhead de estabelecer novas conexões repetidamente.

- Declare conexões fora do handler
- Reutilize em múltiplas execuções
- Reduz significativamente a latência

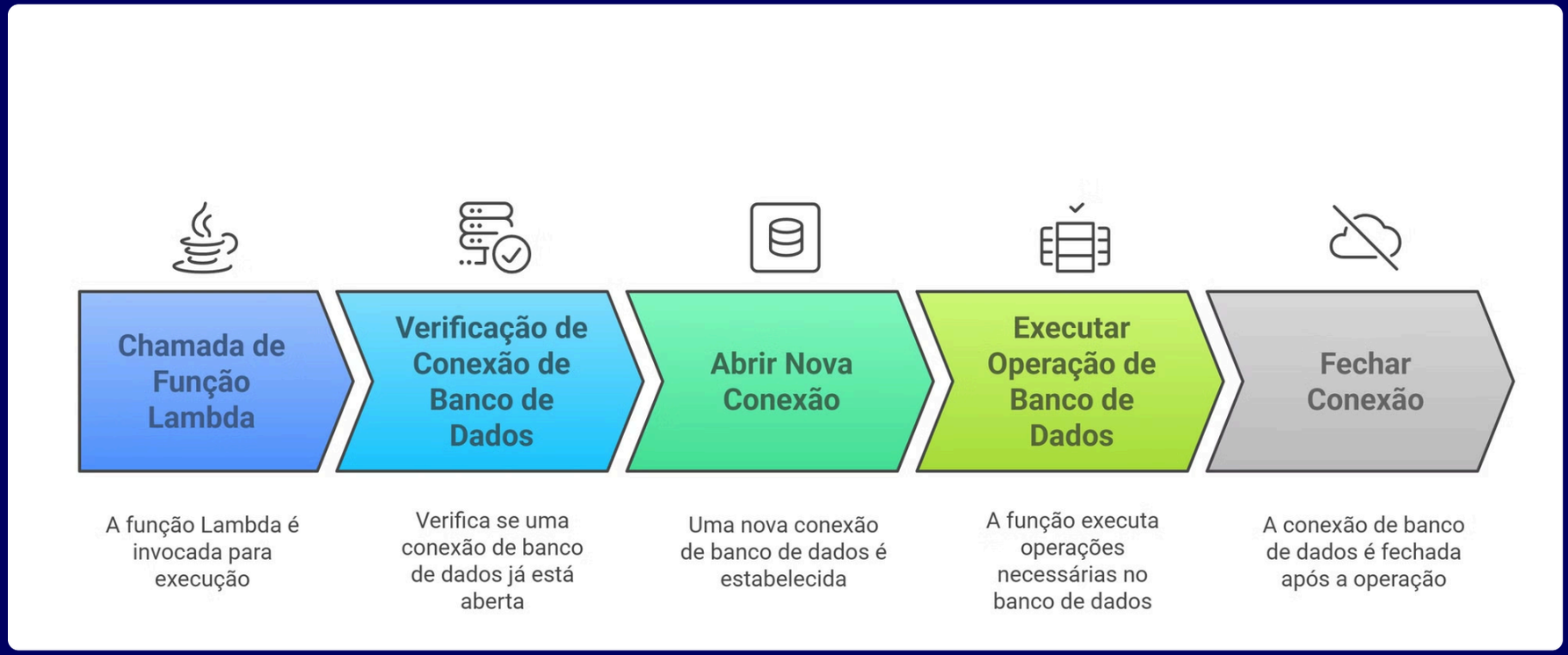
Benefícios

Redução drástica da latência em funções frequentemente invocadas.

- Elimina cold starts parcialmente
- Melhora desempenho geral
- Reduz custos de execução

Sem Persistência de Conexões: Exemplo

Imagine que sua função Lambda precisa falar com um banco de dados. Normalmente, cada vez que sua função é chamada, ela precisa abrir uma nova conexão com o banco de dados. Abrir uma conexão leva tempo, especialmente quando a função está "fria" (cold start) e precisa se preparar do zero.



Com Persistência de Conexões: Exemplo

Conexões persistentes são como deixar a porta do banco de dados aberta. Em vez de fechar a porta (a conexão) toda vez que a função termina, ela a mantém aberta. Na próxima vez que a função for chamada, ela pode usar a porta já aberta (a conexão persistente) e economizar tempo, evitando o processo de abrir uma nova conexão. Isso é especialmente útil depois do "cold start", quando a função já se preparou e quer trabalhar o mais rápido possível.



Triggers: Automatizando a Execução

Eventos

Ações que disparam a execução da função Lambda.

Serviços AWS

Podem ser configurados de serviços como: S3, DynamoDB, SNS, SQS, API Gateway, etc.

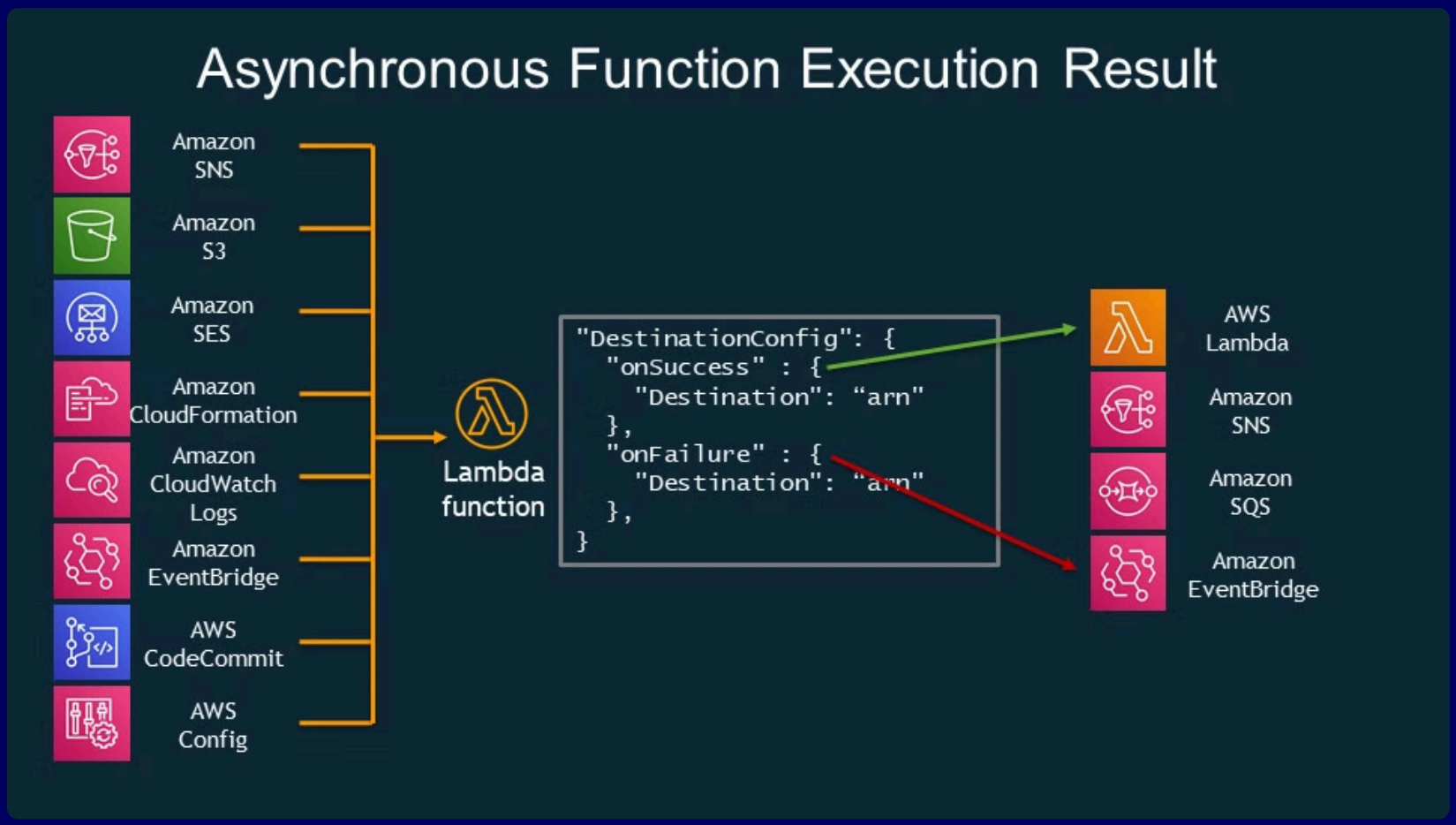
Configuração

Definida no console AWS, CLI, SDK ou CloudFormation.



Triggers permitem que funções Lambda respondam automaticamente a eventos em outros serviços AWS, criando arquiteturas orientadas a eventos.

Destinos: Processando Resultados



SNS (Simple Notification Service)

Envie notificações por e-mail, SMS ou para outros endpoints.

SQS (Simple Queue Service)

Enfileire resultados para processamento assíncrono.

EventBridge

Roteie eventos para outros serviços AWS.

Outra Função Lambda

Encadeie funções para processamento em etapas.

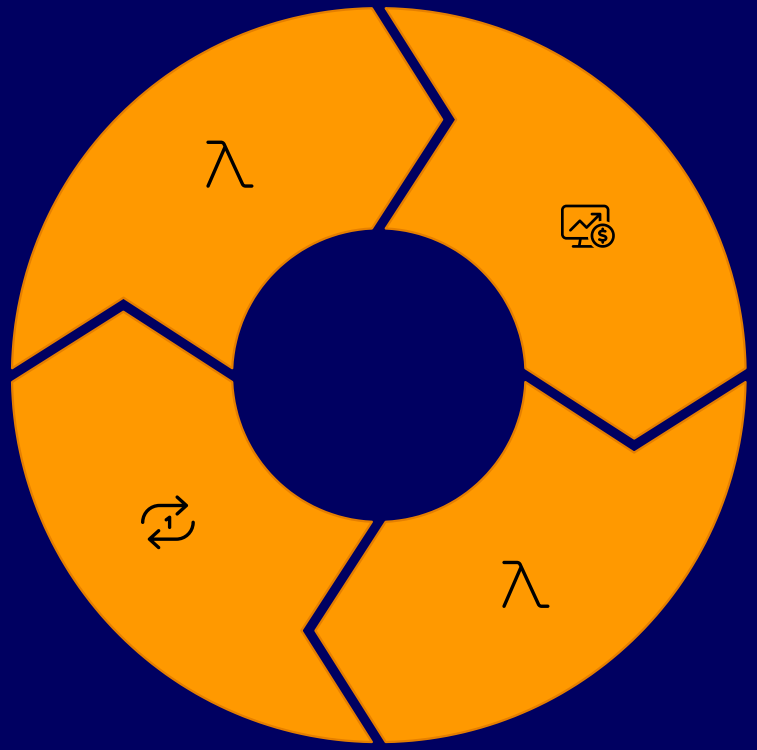
Observação: AWS CodeCommit foi descontinuado e não está mais liberado para novos clientes. Acesse o link para mais informações.

Fonte: https://aws.amazon.com/pt/blogs/compute/introducing-aws-lambda-destinations/

Cuidado com Loops Infinitos

Função A

Processa evento e gera saída.



Serviço Intermediário

Recebe saída e gera novo evento.

Loop Infinito

Saída aciona novamente a Função A.

Função B

Processa novo evento e gera saída.

Configure suas funções Lambda e destinos com cuidado para evitar loops infinitos que podem gerar custos elevados e sobrecarregar seus sistemas.

Integração com DynamoDB Streams

O que são DynamoDB Streams?

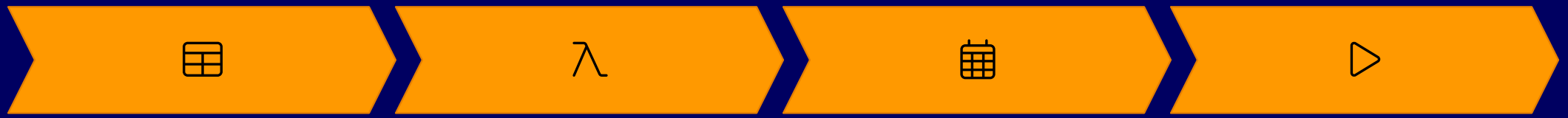
Fluxo ordenado de eventos que representam modificações em uma tabela DynamoDB.

Captura inserções, atualizações e remoções de itens em tempo real.

Tipos de Eventos

- INSERT: Novo item inserido
- MODIFY: Item existente modificado
- REMOVE: Item removido da tabela

Configurando Trigger DynamoDB



Stream Habilitado

Tabela DynamoDB com streams ativos.

Função Lambda

Crie ou selecione uma função para processar eventos.

Configurar Trigger

Adicione o stream como trigger da função.

Testar

Modifique itens na tabela para testar a integração.

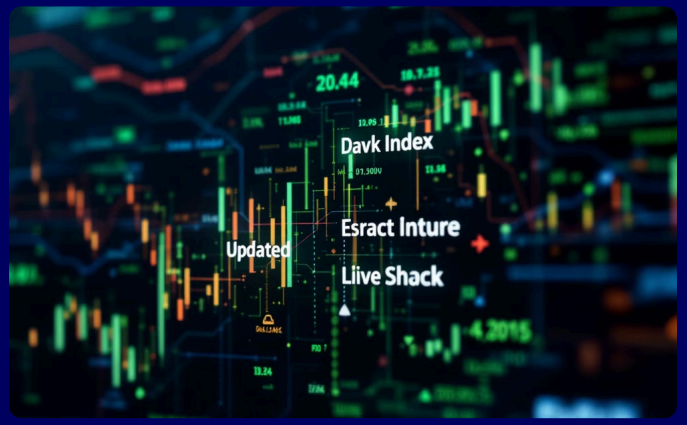
Exemplo: Processando Eventos DynamoDB

```python
import boto3

def lambda_handler(event, context):
    for record in event['Records']:
        if record['eventName'] == 'INSERT':
            new_item = record['dynamodb']['NewImage']
            # Atualize o índice secundário com base nos dados do novo item
            print("Novo item inserido:", new_item)
    return {"statusCode": 200, "body": "Processamento concluído."}
```

Este código é um exemplo de função Lambda que é acionada por um evento do DynamoDB Streams. Quando um novo item é inserido em uma tabela do DynamoDB, o evento é enviado para a função Lambda.

Aplicações Práticas: DynamoDB Streams



Manter Índices Atualizados

Atualizar índices secundários sempre que um item é modificado na tabela principal.



Análise em Tempo Real

Disparar processos de análise quando novos dados são inseridos.



Notificações

Enviar e-mails ou SMS quando registros importantes são modificados.

Por que o que vimos sobre Desenvolvendo e Configurando AWS Lambda é Importante?

(Visão Prática e da Certificação)

Principais Ajustes no Lambda



Runtime

A escolha impacta diretamente no desempenho e tempo de inicialização.



Memória e Timeout

Defina timeout realista para evitar execuções desnecessariamente longas.



Variáveis de Ambiente

Facilita mudanças sem necessidade de reimplantar o código.

Uso de Layers e Concurrency

Layers

Compartilhar dependências e reduzir o tamanho do deployment.

Concorrência Provisionada (Provisioned Concurrency)

Mantém instâncias pré-aquecidas sempre prontas para execução imediata.

Concorrência Reservada (Reserved Concurrency)

Define o limite máximo de execuções simultâneas da função.

Monitoramento

Ajustar concorrência para equilibrar custo e performance.

Triggers e Destinos

Triggers

Automatizar execuções (ex.: eventos do S3, mensagens do SQS).

Destinos

Encaminhar saídas para serviços como SNS e SQS.

Integração

Criar fluxos eficientes entre componentes da arquitetura.

Por que Dominar o AWS Lambda é um Diferencial Competitivo



Otimização de Aplicações Serverless

Configurações adequadas impactam diretamente na performance e escalabilidade das soluções cloud.



Redução de Custos Operacionais

Ajustes precisos de memória, timeout e concorrência evitam desperdícios financeiros na nuvem.



Automação e Integração

Triggers e destinos permitem criar fluxos de trabalho inteligentes com mínima intervenção manual.



Diferencial no Mercado

Empresas buscam profissionais que dominam estes conceitos para soluções escaláveis de alto desempenho.

AWS Lambda na Certificação Developer Associate

Entenda por que dominar o AWS Lambda é crucial para seu sucesso na certificação e na prática profissional

Peso na Prova

Lambda representa um peso relevante nas questões da certificação.

Fundamentos Práticos

A AWS espera que você saiba implementar e ajustar funções Lambda.

Integração Serverless

Questões frequentemente abordam integração com outros serviços AWS.

Troubleshooting

Identificar e resolver problemas comuns de configuração é essencial.

A certificação cobra aspectos práticos como configuração de runtime, memória, timeout e uso de layers.

Conhecimento sobre triggers, destinos e otimização de performance frequentemente aparece nas questões.